Controlling a DC Motor with Delay: Study Notes (EEE8089)

Oct 30, 2026 · @Alex

The big picture

The whole Maxima notebook makes one point: **delay, not the motor, is what limits how hard you can push a control loop**, and PI/PID control is how you still get zero error and a fast response despite it.

These notes follow the notebook section by section, from the simplest block to the full PID loop. They are written for embedded-systems students, not control specialists. You do not need to solve differential equations; you need to read four kinds of plot and know what each coefficient does.

In the coursework (Part 2), the loop is built from Linux processes on Raspberry Pis: `gpiomon` → `encoder.c` → pipe or network → `controller.c` → `/dev/serial0` → driver board → motor → encoder. Every arrow in that chain takes time. The notebook models all of that time as one lumped delay of 1 time unit, and models the motor as a sluggish first-order system 10 times slower than that delay.

How to use these notes:

1. Open the notebook in wxMaxima next to these notes and re-run each cell as you read.
2. Change one number at a time (a gain, the delay) and predict the plot before you look.
3. Answer the self-check questions at the end before the lab session.

Toolkit: just enough to read the plots

Transfer function. A block is described by a ratio output/input written in the variable s. Think of s as "rate of change" (d/dt). Blocks in series simply multiply: delay × motor × controller.

The four building blocks in the notebook:

Block	Maxima expression	What it does in time	Gain on a Bode plot	Phase on a Bode plot
Motor (first-order lag)	`1/(10*s+1)`	Approaches the new speed exponentially, 63% after one time constant τ = 10	Flat, then falls 20 dB/decade above 1/τ	0° down to −90°
Delay	`exp(-s)`	Exact copy of the input, 1 time unit late	Always 1 (0 dB)	−ω radians: grows without limit
Integrator	`1/s`	Accumulates the input (running sum)	Infinite at DC, falls 20 dB/decade	Always −90°
Differentiator	`s`	Responds to the rate of change	Zero at DC, rises 20 dB/decade	Always +90°

Closed loop. `closed_loop(L)` computes what you get when the open-loop chain L is wrapped in unity negative feedback:

```
T(s) = \frac{L(s)}{1 + L(s)}
```

The loop goes unstable when L = −1, i.e. when the signal comes back around the loop with gain 1 and phase −180°, so the feedback adds to the error instead of subtracting from it.

The plots.

- *Bode plot*: gain and phase against frequency. Read the open loop L here to judge stability (margins, see Section 4).
- *Step response*: the output after the reference jumps from 0 to 1. This is what you will measure on the rig.
- *Nyquist plot*: the same frequency response drawn as one curve in the complex plane. Optional for this module.
- `time_delay(1,20)`: a Padé approximation of `exp(-s)`. Step responses need a rational function, so the exact delay is replaced by a 20th-order approximation. Small ripples before t = 1 in step plots are an artefact of this, not physics.

Units. Time in the notebook is measured in delay units (delay = 1). Frequency is in rad per time unit. "From the model to the Raspberry Pi rig" below shows how to convert to seconds and Hz.

Notebook §1: the motor on its own

Result: the motor behaves like an RC low-pass filter. Apply a step of voltage and the speed rises exponentially to its final value; it reaches 63% after one time constant τ and is essentially there after 5τ.

The model is the first-order equation $y' = (x - y)/\tau$, whose transfer function is:

```
G_m(s) = \frac{K}{\tau s + 1}
```

- K is the static gain: final speed per unit of drive (on the rig, Hz per driver-code step).
- τ is set by the rotor's inertia and friction. The notebook normalises K = 1 and τ = 1 for the first plots.
- On the Bode plot the gain is flat up to the corner frequency 1/τ, then falls 20 dB per decade; phase slides from 0° to −90°.

Why it matters: a first-order lag on its own never adds more than 90° of phase. That is why, in the next section, a plain gain around it can be turned up without ever going unstable.

Try it: change `1/(s+1)` to `1/(10*s+1)` and confirm the step response takes 10 times longer and the Bode corner moves 10 times lower.

Notebook §2: the ideal loop (no delay)

Result: without delay, more gain is always better — faster and more accurate, and never unstable. This is the textbook picture, and it is unrealistic.

Put a gain a in front of the motor and close the loop:

```
T(s) = \frac{a}{s + 1 + a}
```

Three things follow directly from this formula:

Gain a	Final value (target = 1)	Steady-state error	Closed-loop time constant
1	0.50	50%	1/2 = 0.5
10	0.91	9%	1/11 ≈ 0.09
100	0.99	1%	1/101 ≈ 0.01

- **Speed:** the closed loop is still first-order, but its time constant shrinks to 1/(1 + a). Feedback makes a slow motor look fast.

- **Accuracy:** a proportional controller always leaves an error of 1/(1 + a). It needs some error to produce any drive at all.
- **Stability:** the phase never reaches −180°, so no gain can make it oscillate.

The catch, which the rest of the notebook is about: every real loop contains processing delay, and delay breaks the "more gain is better" rule.

Notebook §3: the delay on its own

Result: a delay passes every frequency at full strength but turns its phase further and further back, so a loop containing only a delay oscillates once the gain reaches 1.

In the rig the delay comes from the driver board, the 20 ms frequency measurement, the pipes between processes and, in experiment 2, the network. The notebook lumps it all into exp(−s): the output is the input, 1 time unit later.

Bode plot: the gain line is flat at 1, which is why the plot looks uneventful. The phase is the problem: it is −ω radians, so it passes −180° at ω = π and keeps going.

Delay in a loop — walk through it in time. With only a gain a and a delay, each output sample is a × (reference − the output one delay earlier):

1. t = 0 to 1: nothing has come back yet, the error is 1.
2. t = 1 to 2: the output jumps to a; the error becomes 1 − a.
3. t = 2 to 3: the output becomes a(1 − a), and so on: it zig-zags with period 2.

- a = 0.8: the zig-zag shrinks each round and settles at a/(1 + a) ≈ 0.44.
- a = 1.0: the zig-zag never dies — a sustained oscillation between 0 and 1.
- a = 1.1: each round is 10% bigger than the last — unstable.

Key intuition: the controller is always reacting to old news. If it reacts strongly enough to correct the whole error in one go, by the time the correction is seen the situation has moved, and it overcorrects.

Notebook §4: motor + delay under P control

Result: with a delay of 1 and a motor time constant of 10, a proportional gain above about 16 makes the loop oscillate, and any gain low enough to be safe leaves a visible steady-state error. This trade-off is the reason integral action exists.

The open loop is now:

```
L(s) = a \cdot \frac{e^{-s}}{10s + 1}
```

Reading stability off the Bode plot — two margins. Look at the open loop L, not the closed loop.

- **Gain margin:** find the frequency where the phase reaches −180° (here $\omega \approx 1.63$). Read the gain there: about 1/16, i.e. −24 dB. You can multiply the loop gain by about 16 before it hits the edge. The notebook reads this as "about 1/17" from the plot.
- **Phase margin:** find the frequency where the gain is 1 (0 dB). Read how far the phase is from −180°. Rule of thumb: 45° to 60° gives a well-damped response; under about 30° gives large overshoot and ringing.

What the step responses show:

Gain a	Behaviour	Steady-state error 1/(1 + a)
1	Slow, smooth, settles at half the target	50%
10	Overshoot and ringing (phase margin ≈ 39°)	9%
20	Growing oscillation: past the gain margin	— (unstable)

The oscillation that appears near the limit has a period of about $2\pi/1.63 \approx 3.9$ delay units: the loop rings at the frequency where it has −180° of phase. On the rig this is a useful diagnostic — if you see oscillation, its period tells you roughly where your delay is.

The dilemma: P control cannot be both stable and accurate here. We need something with very large gain at DC (to kill the steady error) but modest gain near $\omega \approx 1$ (to stay stable). That is exactly what an integrator offers.

Notebook §5–6: integrator and differentiator

Result: the integrator is the tool for accuracy, the differentiator is the tool for speed and damping, and each costs 90° of phase in opposite directions.

Integrator 1/s. Its output is the running sum of its input. As long as any error remains, the sum keeps growing and pushes the motor harder. The only way it can stop changing is when the error is exactly zero — so an integrator in the loop forces zero steady-state error. The price: a constant −90° of phase, which eats into the margin.

In code (from the lecture flowchart): `I = I + E*Tsample`.

Differentiator s. Its output is the rate of change of its input. It reacts to where the error is heading, not where it is, so it can brake an overshoot before it happens. It gives +90° of phase, which can buy back some margin. The price: gain rises with frequency, so it amplifies measurement noise.

In code: `D = (E - Eold)/Tsample`.

	Proportional Kp	Integral Ki/s	Derivative Kd·s
Reacts to	The error now	The error accumulated in the past	The trend of the error
Main benefit	Basic drive	Zero steady-state error	Damping, faster settling
Phase	0°	−90°	+90°
Main risk	Too high → oscillation	Too high → slow overshoot; wind-up	Noise amplification

Notebook §7: PI control

Result: Kp = 5, Ki = 0.5 is the best of the nine combinations tried — zero steady-state error, about 10 dB of gain margin and about 60° of phase margin. It is no accident that Kp/Ki = 10, the motor's time constant.

The controller is C(s) = Kp + Ki/s. The notebook sweeps Kp ∈ {3, 5, 7} and Ki ∈ {0.3, 0.5, 0.7}, then judges each by two numbers read off the open-loop Bode plot:

1. Gain at the −180° phase point should be well below 0 dB (the notebook asks for about −10 dB).
2. Phase at the 0 dB gain point should be well above −180° (the notebook aims for about −110°, i.e. a phase margin near 70°).

Why Kp = 5, Ki = 0.5 works so well. Rewrite the PI controller with a common denominator:

```
K_p + \frac{K_i}{s} = \frac{K_i\,(\tfrac{K_p}{K_i}s + 1)}{s} = \frac{0.5\,(10s + 1)}{s}
```

The factor (10s + 1) is exactly the motor's denominator, so it cancels. The whole loop collapses to an integrator plus a delay:

```
L(s) = \frac{0.5\,e^{-s}}{s}
```

Now the margins can be worked out by hand: gain is 1 at ω = 0.5, where the phase is −90° − 29° ≈ −119° (phase margin ≈ 61°); phase is −180° at ω = π/2, where the gain is 0.5/1.57 ≈ 0.32 (−10 dB). These match the plot.

A rule of thumb students can reuse:

1. Set the ratio Kp/Ki equal to the motor time constant τ (cancel the slow pole).

2. Then choose Ki (with Kp following) to place the crossover well below 1/delay. With the motor gain normalised to 1, Ki ≈ 0.5/delay is a good start, and Kp = Ki × τ.

Kp, Ki	Kp/Ki = τ?	Loop	Phase margin	Character
3, 0.3	yes	0.3·e^(−s)/s	≈ 73°	Very safe, slow
5, 0.5	yes	0.5·e^(−s)/s	≈ 61°	Good balance
7, 0.7	yes	0.7·e^(−s)/s	≈ 50°	Faster, some overshoot
others	no	Zero misses the motor pole	—	Slow creeping tail (zero too low) or extra overshoot (zero too high)

The Nyquist plots in the notebook show the same story in another form; you can skip them on a first reading.

Notebook §8–9: PID control, and PI vs PID

Result: Kp = 7, Ki = 0.7, Kd = 2.2 crosses 0 dB at about ω ≈ 0.69 instead of 0.5, so it responds roughly 40% faster than the best PI, with a similar phase margin (≈ 63°) and a gain margin of about 8 dB.

The controller is C(s) = Kp + Ki/s + Kd·s. The notebook sweeps Kp ∈ {5, 7}, Ki ∈ {0.5, 0.7}, Kd ∈ {0, 2.2}.

What the derivative term does here. Factor the PID numerator for the best set:

```
7 + \frac{0.7}{s} + 2.2s = \frac{2.2\,(s + 0.103)(s + 3.08)}{s}
```

- The zero at 0.103 again sits almost on the motor pole at 0.1 and cancels it — the same trick as PI.
- The second zero at 3.08 adds phase lead around crossover. That extra phase is what lets the loop gain be raised by about 40% without losing margin.

PI vs PID at a glance (both loops checked by hand from the formulas above):

	Best PI (5, 0.5, 0)	Best PID (7, 0.7, 2.2)
0 dB crossover ω	0.50	≈ 0.69
Phase margin	≈ 61°	≈ 63°
Gain at −180°	≈ −10 dB	≈ −8 dB

	Best PI (5, 0.5, 0)	Best PID (7, 0.7, 2.2)
Steady-state error	0	0
Sensitivity to measurement noise	Low	Higher (derivative)

Should you use D on the rig? Only cautiously. The model's derivative is ideal; on the rig it differentiates a quantised speed measurement (see the next section). Start with PI, get it stable, then add a small Kd and check that the step response improves rather than becoming jittery. Also note the delay limits everything: no choice of coefficients pushes the crossover much beyond about 1/delay.

From the model to the Raspberry Pi rig

Result: the notebook's coefficients are a shape, not a recipe — you must measure your rig's gain, time constant and delay, rescale, and then rerun the notebook with your own numbers.

1. Measure three things in open loop

1. **Motor gain K.** Send a fixed code (e.g. `echo 837 ... > /dev/serial0`, i.e. 100 above the zero code 737) and read the steady speed from `encoder.c`. K = speed ÷ 100, in Hz per code unit.
2. **Time constant τ.** From the same step, the time to reach 63% of the final speed.
3. **Delay Td.** The dead time between sending the code and the first visible change in speed. Repeat over the network for experiment 2.

2. Rescale the coefficients

The model assumed a motor gain of 1 and measured time in units of Td. To use its results on the rig:

- Choose `Km = 1/K` in `Y = 737 + PID*Km`, so the driver-plus-motor gain is normalised back to 1.
- Keep Kp as it is. Convert Ki_rig = Ki ÷ Td and Kd_rig = Kd × Td, with Td in seconds.
- If your τ/Td is not 10, redo the PI rule from §7: Kp/Ki = τ (both in the same time units).

3. Sampling and the encoder

- `encoder.c` counts rising edges of GPIO9 in a 20 ms window. At the 1 Hz target that is 500 edges/s, so only about **10 edges per window**: one edge more or less is a **0.1 Hz (10%) jump** in the reading.

- The window itself adds delay (the reading describes, on average, a moment about 10 ms ago). Include this in Td.
- That quantisation is why D is risky: a one-edge flicker gives a derivative spike of 0.1 Hz ÷ 0.02 s = 5 Hz/s, multiplied by Kd. Options: a longer window, period measurement instead of counting, or a simple low-pass filter on D.
- Use the real elapsed time as dt in `I = I + E*dt` and `D = (E - Eold)/dt`; the slides remind you not to forget it.

4. Limits and wind-up

- Clamp the output to the valid range 737 ± 600 before writing to the serial port.
- While the output is clamped, stop adding to I (anti-wind-up). Otherwise the integrator keeps growing during saturation and causes a large overshoot afterwards.
- Stay within the recommended ±10 Hz: above that the interrupts cannot keep up and the measurement becomes wrong.

5. A ready-made hypothesis for experiments 1 and 2

For a well-tuned PI loop the phase margin is roughly 90° minus (crossover frequency × delay). Extra delay ΔT eats ωc × ΔT radians of margin. For the best PI design (ωc = 0.5/Td, margin ≈ 61° ≈ 1.06 rad), the loop should tolerate about **2 × Td of extra delay** before oscillating.

Testable prediction: moving the encoder to another Pi (experiment 2) increases overshoot and ringing; if the added network delay exceeds about twice the local delay, the same gains go unstable. Network delay also varies (jitter), so expect less repeatable step responses.

6. Capturing the step response

Alternate the reference between 0 and 1 Hz with a period several times the settling time, pipe the encoder output through `tee samples.dat`, then `plot 'samples.dat' with lines` in gnuplot. Overlay your measurement on the notebook's step response with the rescaled time axis — agreement is the best evidence your model is right.

Summary and self-check

The story in five lines:

1. The motor is a slow first-order lag; feedback alone would make it fast and accurate.
2. Delay adds phase lag without limit, so it caps the usable gain (about 16 for τ/Td = 10).
3. P control is then stuck between oscillation and steady-state error.

4. PI removes the error; choosing Kp/Ki = τ cancels the motor lag and leaves a clean integrator-plus-delay loop.
5. PID adds phase lead for about 40% more speed, at the cost of noise sensitivity.

Self-check questions (answers are all in the notes above):

1. Why does the delay's Bode gain plot look "boring", yet the delay is the cause of instability?
2. A P-only loop with a = 10 settles at what fraction of the target?
3. In a gain-plus-delay loop with a = 1, what is the period of the oscillation, in delay units?
4. Why can an integrator only stop changing when the error is exactly zero?
5. Show that Kp = 7, Ki = 0.7 also cancels the motor pole. What changes compared with Kp = 5, Ki = 0.5?
6. Your measured delay is 40 ms. Convert Kp = 5, Ki = 0.5 to rig units.
7. At 1 Hz, how many encoder edges arrive in a 20 ms window, and what does that mean for Kd?
8. Predict what happens to overshoot when the controller and encoder run on different Pis.

Glossary

Term	Meaning
Open loop L(s)	Everything around the loop multiplied together: controller × driver × motor × delay
Closed loop	Output/reference with feedback: L/(1 + L)
Time constant τ	Time for a first-order system to reach 63% of a step
Crossover frequency ωc	Frequency where the open-loop gain is 1 (0 dB); sets the response speed
Phase margin	180° + phase at ωc; 45°–60° is comfortable
Gain margin	How much the loop gain can rise before the −180° point reaches gain 1
Steady-state error	Error left after the response settles
Padé approximation	Rational stand-in for exp(−sT), used in `time_delay`
Wind-up	Integrator growing while the output is saturated, causing overshoot later